Computers & Security 154 (2025) 104458

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

DecoyPot: A large language model-driven web API honeypot for realistic attacker engagement

Anıl Sezgin [a,*], Aytuğ Boyacı [b]

[a] Research and Development, Siemens A.S, Istanbul, 34870, Turkey
[b] Department of Computer Engineering National Defence University, Air Force Academy, Istanbul, 34149, Turkey

ARTICLE INFO

Keywords:
Artificial intelligence
Large language model
Cyber security
Honeypot
Decoy
Deception

ABSTRACT

As cyberattacks get more sophisticated, security systems must learn to detect and deceive them. DecoyPot, a honeypot Web Application Programming Interface (API) that generates legitimate API responses, is introduced in this paper. DecoyPot's command extractor module carefully analyzes API requests to create prompt-response pairs that improve a Retrieval-Augmented Generation based (RAG) large language model (LLM). DecoyPot can instantly adjust its answers to mimic API activity in a contextually correct and convincing manner to attackers. To assess system efficacy, we used a two-phase similarity analysis. Initial queries were matched with prompt-response pairs to ensure contextually suitable responses. Second, similarity measures were used to compare generated responses to reference responses, producing an average score of 0.9780. The high score shows that the system can create API-like responses, boosting its utility. DecoyPot engaged opponents and learned their Tactics, Techniques and Procedures (TTPs). The study shows that honeypot cybersecurity effectiveness must be improved by merging AI-driven response creation with enhanced deception technologies. DecoyPot effectively adapts to incoming queries and generates API-like responses, delivering actionable cyber threat intelligence and enhancing proactive defense strategies.

1. Introduction

Over the last couple of decades, the digital landscape has completely changed, and cyberattacks have increased in all sectors. These range from small organizations to critical infrastructure, exploiting vulnerabilities brought about by the proliferation of connected devices. This rapid growth of attack surfaces automatically demands innovation and adaptiveness in cybersecurity measures. One critical tool among these is the honeypot. Honeypots operate by emulating vulnerable systems to attract attackers and divert their focus from real targets, thereby revealing their techniques. This type of interaction generates valuable intelligence that informs the security strategy, discloses patterns of attack, and enables countermeasure development.

Cybersecurity honeypots have evolved significantly from their initial designs, which were limited to static systems aimed at identifying and logging basic network activity. While these early implementations provided valuable insights into basic attacker behaviors, they were insufficient for addressing sophisticated threats, such as advanced persistent threats (APTs). Modern honeypots have transitioned into dynamic and interactive systems that simulate realistic environments, enabling real-time engagement with attackers and capturing detailed information about their tactics, techniques, and procedures. Despite these advancements, high-interaction honeypots present challenges, including increased complexity, resource demands, and the need for strict isolation to avoid potential risks to production environments (Kristyanto et al., 2022).

Despite their advancements, traditional honeypots face significant challenges in adapting to the rapidly evolving tactics of modern attackers. Static or poorly configured systems are particularly vulnerable to detection, causing attackers to disengage or, worse, exploit the honeypot to mislead defenders. High-interaction honeypots, while more effective at engaging attackers, generate large volumes of data that require time-intensive manual analysis, leading to response delays and missed opportunities for threat mitigation. Furthermore, many organizations struggle to operationalize the intelligence gathered from these systems, limiting their overall utility (Jiang and Zheng, 2020). Traditional honeypots often fail to attract sophisticated attackers, such as state-sponsored adversaries, due to their lack of dynamic interaction, which restricts their ability to gather actionable intelligence on attacker motivations and goals.

* Corresponding author.
E-mail address: anil.sezgin@siemens.com (A. Sezgin).

https://doi.org/10.1016/j.cose.2025.104458
Received 29 October 2024; Received in revised form 11 February 2025; Accepted 21 March 2025
Available online 22 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Integration of artificial intelligence (AI) and machine learning (ML) in honeypot architecture has brought a sea change in the traditional designs that were usually challenged by many drawbacks. This is achieved by utilizing AI for network traffic analysis and the identification of malicious behavior, followed by adaptive adjustment of their responses to keep the attackers engaged. The capabilities would make honeypots more resilient and increase their efficacy in detecting and countering complex threats. Machine learning allows honeypots to filter out background noise and irrelevant data efficiently, thus focusing on real threats. This reduces the workload for analysts, accelerates detection and response, and overall reinforces the cybersecurity framework (Angeli et al., 2024).

Recent advancements have enhanced honeypots by integrating large language models, including GPT-3. This model allows honeypots to interact with adversaries in a very human-like way, therefore luring them into exposing their tactics, techniques, and procedures. These interactions are particularly useful for responding to social engineering attacks, where the adversary leverages human vulnerabilities to gain unauthorized access to systems. In such cases, LLM-enhanced honeypots gather valuable intelligence on attacker tactics and motivations by generating realistic dialogues (Raut et al., 2023). The latest generation of honeypots, equipped with retrieval-augmented generation techniques, offers greater dynamism and adaptability. RAG allows honeypot systems to achieve a far better level of contextual accuracy and responsiveness by leveraging external data sources for real-time adaptation to diversified attack patterns and conditions.

The integration of RAG and LLMs brings substantial novelties to honeypot design: first, RAG frameworks allow the honeypot to fetch relevant external data with which to generate contextually appropriate responses, enhancing the realism in emulating a real-world environment (Dang et al., 2024). In a honeypot, it enhances the reality enough to make it credulously sustainable for extended interaction with attackers. Similarly, LLM allows for complex and dynamic responses that keep pace with adversaries' expected actions strongly. In the context of web application honeypots, advanced technologies have been utilized in emulating API operations in an efficient manner while their outcomes are based on contextual information. Therefore, combining RAG and LLMs arms honeypots with a capability of high-order deceiving while harvesting critical intelligence of attacker's techniques and strategic mindsets (Buzzio-Garcia, 2021).

This paper presents a new honeypot system using large language models with retrieval-augmented generation to mimic the conversations involved in Web APIs. The first part outlines two important components: the command extractor and the response generator. The command extractor converts the API requests into structured prompt-response pairs for enriching the fine-tuning of the large language model. In contrast, a response generator augmented with out-of-the-box knowledge from retrieval-augmented generation would simulate realistic and contextually relevant responses to engage attackers. The system was evaluated using a best-match similarity metric, whereby the inputs from attackers were compared with outputs produced by the LLM. The test shows that the honeypot is able to realistically emulate a real system and attract malicious actors, providing real-world information to improve security measures.

This study introduces DecoyPot, a novel honeypot system that leverages state-of-the-art technologies, including Retrieval-Augmented Generation and Large Language Models, to create dynamic and contextually accurate API interactions. Unlike traditional honeypots, DecoyPot combines generative modeling with retrieval mechanisms to enhance deception capabilities and adapt to sophisticated attacker strategies. The system's effectiveness has been demonstrated through rigorous evaluations, achieving high-response similarity scores with real API interactions. This performance highlights DecoyPot's ability to generate realistic and semantically aligned responses, ensuring a convincing engagement with attackers while collecting valuable intelligence on their tactics and techniques. DecoyPot's architecture has been validated across diverse API domains, including Internet of Things, finance, healthcare, and e-commerce. This adaptability underscores its versatility in addressing various attack vectors, making it a robust tool for proactive cyber defense and deception.

The rest of this manuscript is organized as follows: Section 2 presents related work on honeypot systems, focusing on the progress of research work at the areas of AI and ML integrations. Section 3 describes the design and architecture of the proposed system and highlights the novelties in the work. Section 4 outlines data collection and training methods, along with incorporating a variety of API domains. Section 5 provides an overview of the experimental setup and the evaluation metrics. Section 6, the concluding section, is a detailed discussion of the results and the analysis that followed while Conclusion Section 7 discusses future directions for research.

2. Related work

From very simple decoy mechanisms to more sophisticated, dynamically adapting, intelligent platforms, honeypots have evolved into critical components of modern cybersecurity methodologies. Effectively attracting, interacting with, and misleading attackers, honeypots provide the precious intelligence about malicious strategies besides functioning as an important layer in security. The section is focused on honeypot technology evolution through detailed application strategies, optimization techniques, and applications across all main areas of cybersecurity.

2.1. Introduction to honeypots

Honeypots are security mechanisms that mimic a vulnerable system with the intention of attracting an attacker, studying their behavior, and gathering relevant intelligence. The first honeypots were very simple and static structures that mimicked a common vulnerability, but they proved to be no match against the ever-evolving sophistication of cyber threats. While these primitive honeypots were good enough for simple reconnaissance, their utility quickly faded as attackers developed tools that could identify and avoid these traps (Kristyanto et al., 2022, Angeli et al., 2024, Maldonado et al., 2024).

Such shortcomings were, until recently, countered by new honeypot technologies based ML, AI, and LLMs. These enhancements further allowed the honeypot to dynamically adapt to the new methods of attack, simulate real environments, and interact more effectively with an attacker. The evolution has so far expanded their use beyond the traditional network security to also involve those on Internet of Things (IoT), Industrial Control Systems (ICS), and Cyber-physical systems (CPS) applications (Sladić et al., 2024, Suhail et al., 2023).

2.2. Interaction levels in honeypots

Low-interaction honeypots can mimic very basic functionalities, such as simple network services, to attract attackers. These systems are generally lightweight and very easy to deploy, and hence suitable for detecting reconnaissance activities (Kristyanto et al., 2022). However, their limited interaction capabilities often fail to deceive sophisticated attackers who may identify them as decoys. The study in (Lygerou et al., 2022) illustrates the use of low-interaction honeypots in IoT networks. Such systems emulate protocol-specific environments to monitor IoT traffic to detect potential threats (Karpukhin et al., 2020). Although useful for the initial detection stage, low-interaction honeypots cannot provide an insight into advanced attack strategies.

Medium-interaction honeypots provide an effective compromise between functionality and resource efficiency by emulating services such as databases or web servers. These systems are particularly suitable for environments with moderate security requirements, as they balance engagement depth with ease of deployment and maintenance (Karpukhin et al., 2020, Lewis et al., 2020). By enabling attackers to

interact with emulated services, medium-interaction honeypots gather detailed intelligence on attacker methods and tools. As an example, a database honeypot emulating a MySQL server would reveal very valuable insights into the SQL injection techniques that a hacker may employ and yet the resource requirements are manageable. This approach is further improved by the LLMs integration, and this is illustrated in (Hu et al., 2024), with training LLMs on SQL queries responses. These honeypots emulate a MySQL database that is vulnerable in order to respond appropriately given real-world contexts. This realism not only effectively fools the attackers but also provides nuanced data about their tactics, thus making the medium-interaction honeypot a practical and insightful tool in cybersecurity strategies.

High-interaction honeypots emulate whole systems, allowing the attacker free rein to interact with a realistic environment. These are resource-intensive systems but very valuable for research into advanced threats in critical infrastructure sectors such as ICS (Lygerou et al., 2022, Rashid et al., 2024). In high-stakes environments like ICS, high-interaction honeypots must accurately replicate industrial processes to gather actionable intelligence on attacker behavior. For example, works like (Lygerou et al., 2022) consider using game-theoretic models to optimize the deployment of high-interaction honeypots while ensuring efficient resource allocation for maximizing threat data collection.

2.3. Deployment strategies

Deployment has significantly changed with honeypots, reflecting the growing complexity of the threat landscape. Advances in deployment strategies have made them more effective and adaptive.

Traditional honeypots are static deployments that emulate vulnerabilities for the purpose of collecting intelligence. They actually fare quite well in monitoring malicious activity in an effort to improve overall network security measures (Kristyanto et al., 2022, Angeli et al., 2024, Maldonado et al., 2024). While effective against simple threats, traditional honeypots have some limitations due to their inability to dynamically react to new attack vectors.

Adaptive honeypots are a giant leap forward, integrating ML and AI in analyzing incoming traffic and dynamically adjusting responses. This learning from attacker behavior extends the engagement time and improves intelligence collection (Gillick et al., 2019). The idea of reinforcement learning, for example, can enable honeypots to adjust their operation with respect to the behavior being observed by an attacker. This is useful, especially in complex attack environments such as enterprise networks and IoT ecosystems. There is a need to keep their pertinence in response to evolving cyber threats by using adaptive honeypots.

Specialized honeypots are thus designed to meet very specific security challenges within certain environments. For example, (Zhou et al., 2023) uses game-theoretic methods to optimize the deployment of honeypots within ICS environments. Similarly, honeypots in the cloud-as discussed in (Rashid et al., 2024), scalable solutions that monitor industrial networks and further secure Industry 4.0 infrastructures.

2.4. Optimization techniques in honeypots

Machine learning and AI have transformed honeypots into systems that can analyze huge amounts of data, identify patterns, and respond on a real-time basis. This enhances the detection capability and reduces manual effort in monitoring and analysis. For example, in the web-based honeypot in (Jiang and Zheng, 2020), ensemble ML techniques are applied to filter out noise and focus on significant threats for improving the accuracy of detection. Similarly, Generative Adversarial Networks (GANs), applied in (Ding, 2024), optimize honeypot responses, giving them greater effectiveness in resource-constrained environments. AI-powered honeypots largely improve the level of interaction by emulating human-like interactive capabilities that enable effective deception and intelligence gathering. Realistic responses of generative AI models in (Raut et al., 2023), let honeypots dynamically learn from and adapt their tactics to attackers.

The deployment of honeypots is optimized using behavioral intelligence from the SOAR engine (Bartwal et al., 2022), analyzing attacker behaviors to guide strategic placement. When deployed in response to specific threats, these honeypots achieved longer engagement times and captured much more information about attacker tactics. These SOAR findings, in combination with the capabilities of LLMs, make this honeypot especially relevant to modern contexts via the capability of performing and dynamically updating responses in web API settings. The approaches described in (Li et al., 2024) use RAG to increase the realism of honeypot interactions. The RAG-based approach retrieves pre-existing responses from a knowledge base and uses them as contextual inputs during the generation process, enabling the honeypot to correctly mimic the behaviors of legitimate APIs. Combining RAG with LLMs ensures that the responses of the honeypot are contextually accurate and semantically aligned, making it much more effective in engaging and deceiving attackers.

Reinforcement learning is now one of the essential tools in optimizing honeypot functionality in dynamically adjusting the behaviors of an attacker, thereby further extending the durations of engagement. This adaptability enables the honeypot to harvest full intelligence about tactics employed by attackers and raises the utility value of use in the context of cybersecurity dramatically. In this context, (Lopez-Yepez and Fagette, 2022) in the dynamic scenario, dynamic Secure Shell (SSH) honeypots make use of deep reinforcement learning methods for them to become sensitive, intelligently reacting to the behavior of detected attackers. Not only does this customized approach increase the attacker's time of interaction, but it also improves the honeypot ability of collecting vital information and learning new attack schemes. Building on that idea, (Gao et al., 2024) presented DShopping, an intelligent deception system to assist in fighting APTs by relying on exploiting static security configurations.

By using reinforcement learning, DSHopping is able to dynamically change the scenarios of deception for optimized engagement of attackers and real-time detection of APT behaviors. It underlines not only the synergy between machine learning and honeypot systems but also how they grow and react against complex and variable attack vectors. Through dynamic adaptation to adversaries' strategies, the effectiveness of reinforcement learning—powered honeypots is enhanced several times over, while simultaneously contributing exponentially to cybersecurity intelligence. Hybrid AI architectures, fusing deep reinforcement learning, rule-based models, and LLMs, enable further evolution in reinforcing autonomous cybersecurity defense. These architectures, in their basic form, deploy agents in simulated environments that learn to interact with the main purpose of critical network protection by continuously enhancing their capability to predict attacks and response; an example of the adaptability of modern AI-driven honeypots. The incorporation of LLMs within such a hybrid architecture implies that honeypots can enhance the quality of response by better exploiting adaptive learning informed by intricate attack patterns (Loevenich et al., 2024).

Hybrid models represent a big step forward in the design of honeypots, which integrate various optimization techniques to make them more adaptable and scalable. For example, dynamic honeypots as in (Tang et al., 2023). apply deep learning on encrypted malicious traffic to offer better detection and engagement with sophisticated attackers (Suhail et al., 2023). introduces gamified digital twin integration with AI for securing CPS environments and at the same time offering a training platform for security analysts.

This allows an expansion in the application of digital twin technology for a sophisticated honeypot in driverless vehicles. The research work in (Liatifis et al., 2024), elaborates on a simulated complex vehicular system in order to attract an attacker and provide critical data

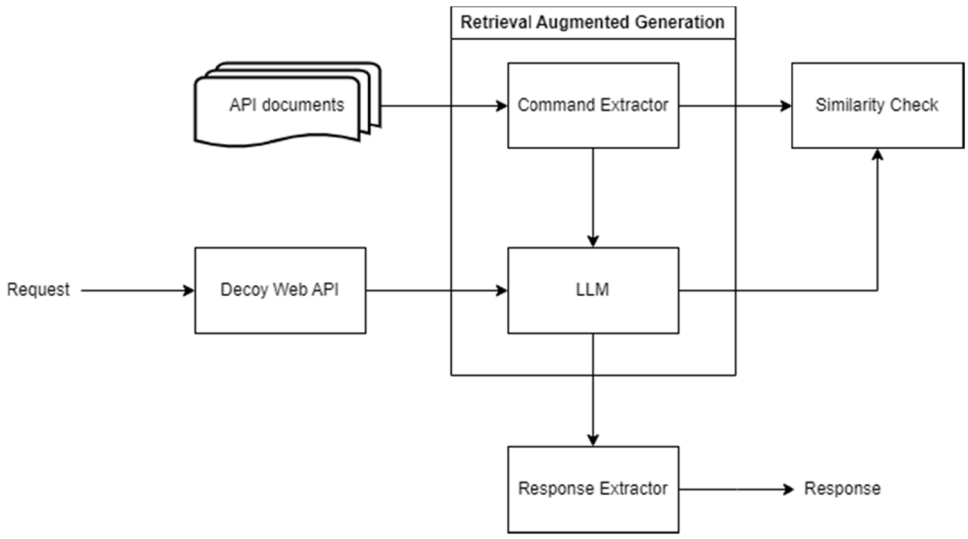

Fig. 1. System architecture.

regarding the attackers' methods. The effectiveness of this method will allow it not only to detect the threats but also adapt quickly to the dynamic threat landscapes. Collectively, these hybrid models give proof that combining deep learning with the concept of gamification, among others, into digital twins could give versatile and resilient honeypot systems.

2.5. Applications in advanced domains

IoT devices are very susceptible to cyberattacks because of their weak security features and easy availability. IoT network-based honeypots use deception technologies in analyzing attackers' behavior for improving the security of devices. Various works, such as (Sharma et al., 2023) and (Qin et al., 2023) present the utilization of IoT-based honeypots for monitoring traffic and detecting threats that target specific protocols. Decentralized honeypots have been deployed in (Karpukhin et al., 2020), to emulate protocol-specific environments, thereby gaining insights into attack patterns and improving IoT security.

Cyber-physical systems combine physical processes with computational systems and thus are a very significant target for sophisticated cyber-attacks. Honeypots in CPS environments are an essential tool in the study and mitigation of such threats. The research in (Tian et al., 2019), puts forth a model that strikes a balance between low- and high-interaction honeypots toward effectively protecting CPS networks. Further, game-theoretic approaches in (Zhou et al., 2023) optimize honeypot deployment in ICS environments toward ensuring efficient use of the honeypot resources while raising detection rates.

Unmanned aerial vehicles (UAVs) rely on interconnected systems, making them susceptible to cyber threats. Honeypots designed for UAV networks mitigate denial-of-service (DoS) attacks by simulating attacker-defender interactions using game theory (Miao and Li ve Q. Pan, 2023). Cloud-based honeypots, as explored in (Rashid et al., 2024), provide real-time insights into threats targeting UAV systems, enhancing their resilience against cyberattacks.

The integration of honeypots and AI-driven IDS in IoT blockchain networks enhances their overall security. In (Commey et al., 2024), researchers employed Bayesian game theory combined with reinforcement learning to dynamically adapt honeypots to the attacker behavior, thus ensuring robust proactive defenses.

Very specialized honeypots are needed in protection ICS environments, since those complex systems under study must be emulated. The importance of high-interaction honeypots, explained in detail in (Zhou et al., 2023), allows deeper studies on attacks against ICS and shows how the most sophisticated attacks can be realized by advanced adversaries. Honeypots of this nature imitate entire systems, enabling extended interactions with attackers, which opens up the possibility for researchers to gather large amounts of data regarding the methodologies, strategies, and processes involved in attacks. In addition, as demonstrated in (Rashid et al., 2024), the scalability and flexibility of cloud-based honeypots make them more useful in industrial environments since they can easily be deployed in different environments. The price paid for such effectiveness in the study of cyberattacks against critical infrastructures is a high resource cost. High-interaction honeypots require detailed configuration to emulate real-world ICS environments, hence their deployment is resource-intensive and technically challenging (Conti et al., 2022). While they are technically difficult to deploy and maintain, due to the fact that they can provide detailed intelligence on cyber threats, they are indispensable in securing critical industrial systems.

3. System design and overview

The decoy Web API honeypot system uses a Retrieval-Augmented Generation framework paired with a fine-tuned large language model to generate responses that are contextually appropriate and semantically plausible. In this architectural design, the system tries to mimic real API functionality as much as possible in order to mislead potential attackers while harvesting valuable information about their strategies. The system has modular architecture, which improves the automation, scalability, and accuracy of the simulation of the API interactions.

3.1. Retrieval-augmented generation framework

RAG is a hybrid model: It combines the precision of retrieval-based systems with the flexibility of generative modeling. Traditional generative models, while impressive in terms of generating human-like text, are generally constrained to patterns learned during training and can therefore easily respond with out-of-context or irrelevant answers; RAG overcomes this limitation. Thus, it uses dynamic retrieval of relevant information from an external knowledge source to ground the generated output in verifiable documentation and ensure responses closely align with real API capabilities.

The RAG framework is operating on two successive phases:

1. **Retrieval Phase**: The system performs a search in a knowledge base of embeddings of API documentation for entries semantically similar to the incoming API request. Embeddings are precomputed via a

sentence transformer model capturing the semantic nature of every interaction with the API from the documentation.

2. **Generation Phase**: The retrieved entries, along with the incoming request context, are fed into the LLM. This dual input ensures that the generated responses maintain coherence with the documented API behavior, producing outputs that align with attackers' expectations.

This mechanism would allow the honeypot to adapt to different API endpoints, thus increasing its effectiveness in deceiving attackers and logging their interaction methods.

3.2. System architecture

The architecture of this system encompasses several interdependent modules, each contributing something unique to the fidelity of the decoy system. The design ensures automation of prompt-response generation, optimization in retrieval, and realistic API emulation. The system architecture is presented in Fig. 1.

Command Extractor module processes API documentation to extract critical details such as HTTP methods, paths, parameters, and expected responses. For instance, a request for a smart home automation platform might include:

Request: *Method:POST;Path:/api/light-control;Data={'deviceId': 'LIGHT001', 'action': 'turn_on', 'brightness': 75}*

Response: *200 OK; {'deviceId': 'LIGHT001', 'status': 'on', 'brightness': 75}*

The extracted details are converted into structured prompt-response pairs, which are used to fine-tune the LLM.

The system's retrieval component performs semantic searches on the knowledge base to find contextually relevant entries through pre-computed embeddings of the API documentation to enable fast and accurate similarity searches. Once a related entry has been retrieved, the fine-tuned language model incorporates information from that with the request received to generate responses compliant with the specifications of the API. This ensures that the generated responses are in the correct formats and are succinct, thereby maintaining the appearance of real API interactions. To validate these responses, a BERT-based similarity model compares them against real API outputs, gauging correspondence through similarity scores. Those that meet the set thresholds will be deployed, while those that do not will be flagged for retraining or refinement to guarantee high fidelity. Finally, API response formatter enhances the outputs by removing unnecessary metadata in order to ensure a homogeneous structure and improve the capacity of the honeypot to convincingly mimic the real API.

The system automatically generates prompt-response pairs for various APIs by continuously updating the knowledge base with newly documented interactions, thus supporting scalability. Hence, the honeypot can handle diverse API endpoints while maintaining high response quality, and its architecture is flexible enough to adapt to these evolving attack vectors with very good long-term relevance.

4. Data collection and training

The quality, diversity, and representativeness of the data used to train a honeypot system largely determine its effectiveness. The training dataset is based on API documentation, complemented with methods to guarantee the observation of privacy standards and preserve structural integrity. This careful calibration further enhances the capability of the LLM to produce realistic API responses.

4.1. API documentation as training data

API documentation, often automatically generated by tools like Swagger or OpenAPI, contains the detailed description of what each endpoint does, including HTTP methods, parameters, and expected responses. This information is then transformed into structured prompt-response pairs, which become the knowledge base.

The dataset used in this study was created from Swagger JSON files originating from various internal projects. These files provided comprehensive API documentation, including HTTP methods, endpoints, parameters, and expected responses. This dataset reflects real-world API interactions and was carefully curated to ensure representativeness across diverse domains, such as IoT, healthcare, finance, and e-commerce. To maintain data integrity and security, sensitive information within the Swagger files was anonymized. This involved replacing identifiable information with placeholder tokens while preserving the structural patterns necessary for accurate model training. The use of internally generated Swagger files ensures the dataset's relevance to real-world applications while adhering to data privacy standards. This dataset is unavailable for public use due to privacy and proprietary constraints but can be accessed upon reasonable request. This carefully designed dataset allows DecoyPot to emulate realistic API behaviors, ensuring its ability to adapt to various domains and effectively deceive attackers targeting specific systems.

This dataset contains an overview of API documentation from various fields, including a large number of application cases to ensure a broad training and robustness. This diversity, in this sense, will enhance the generalizability of the honeypot across different API designs, and it thus becomes highly adaptable to a great variety of attack schemes.

Another big domain covered in this dataset is e-commerce APIs; these would describe critical tasks involving order processing, product catalog management, and payment flows. Many of these APIs often relate to each other or have a hierarchical structure of data, making for a complex setup that the honeypot has to emulate in sets of interactions realistically. Emulating such workflows effectively means emulating common vulnerabilities in online retail platforms.

Another critical domain is APIs for home automation, which is a core in commanding IoT devices within smart home settings. These APIs handle operations such as device control (e.g., adjusting thermostat settings or turning lights on/off) and managing user access to ensure security. Their inclusion in the dataset would have the honeypot simulate complex interactions typical of IoT systems and hence make sense to the attackers targeting smart home infrastructures.

The dataset also involves health care APIs that are responsible for very sensitive operations like handling patient records, prescription, and scheduling. Such APIs are subject to strict regulatory requirements regarding the security of data privacy and integrity. Training the honeypot on such APIs thus helps the latter mimic systems which handle sensitive personal and medical information and are hence targeted for exploitation.

The Financial systems are of high-value area, and the Finance APIs are part of Training data, which support account management, fund transfers, running of fraud detection, among other countless functions, allowing the honeypot to emulate secure financial workflows and reproduce vulnerabilities in the transactions validation, user authentication, or fraud prevention systems exposed in other commonly attacked operations.

Lastly, this dataset includes APIs for student management: these are actually widely used in educational institutions to store student records, course registrations, and academic performance. The honeypot can be trained on these, increasing its capability in mimicking educational administrative systems in detecting and neutralizing various forms of attacks against sensitive educational data or disruption of institutional processes.

The training data set will allow knowledge extraction over different API domains, allowing exploration of different workflows, data architectures, and interaction modalities common in different application scenarios. This will ensure that the honeypot is always effective at dynamically adapting to changing attack vectors, no matter which domain they may target. Diversity within the training data thus leads to greater flexibility, which enhances the system's reliability as a tool for cybersecurity.

Sensitive data in API documentation is masked to meet data privacy and security regulations. Masking replaces sensitive information with generic placeholders, preserving structural patterns while protecting user data. For instance:

Original Data: *{'deviceId': 'LIGHT001', 'location': 'Living Room', 'brightness': 75, 'userId': 'USER001'}*

Masked Data: *{'deviceId': '<DEVICE>', 'location': '<LOCATION>', 'brightness': ⟨BRIGHTNESS⟩, 'userId': '<USER>'}*

This process ensures that the LLM learns only generalized patterns, mitigating the risk of data leakage.

4.2. Fine-tuning the LLM

The fine-tuning process exploits the RAG framework to fine-tune the LLM for API-compliant response generation. The process basically involves the creation of a knowledge base with pairs of prompts and responses collected from API documentation. Every knowledge base entry has a single prompt that captures a pattern in API requests, along with its corresponding response that encodes the expected output, to form the base for contextual grounding during the generation process.

It will encode incoming API requests into embeddings and compare them with the preprocessed and indexed embeddings within the knowledge base. In this way, the system chooses the most contextually relevant entries with the use of cosine similarity and always shows that information which is much closer to the incoming request. These entries form the input context for the LLM, synthesizing responses with the same contextual behaviors that would be documented in API behavior. In this way, it grounds coherently and minimizes mismatched responses that would hint at the honeypot being artificial to attackers.

A similarity validation module, which makes use of a BERT-based model to check on semantic alignment, verifies the generated responses against the documented API outputs. Responses scoring at or above the threshold for similarity are regarded as credible and are deployed, whereas those scoring beneath the threshold are flagged for refinement or excluded from use. The system uses this rigorous approach to validation, ensuring that its honeypot interacts realistically and contextually every time.

4.3. Tools and techniques

The similarity validation module for the honeypot system, spanning diverse fields such as IoT home automation, finance, health, and student management, leverages a robust computational framework. This framework integrates pre-trained transformer-based embeddings with advanced similarity metrics to ensure semantic coherence between the generated responses and authentic API behaviors. By maintaining this high degree of alignment, the system enhances its credibility in accurately emulating real-world API interactions across multiple domains, reinforcing its effectiveness in deception and intelligence gathering.

System uses a BERT model by default, namely bert-base-uncased, as a core component for generating dense semantic embeddings of textual data. With contextual relationships of API requests and responses easily obtainable, the system can encode both the incoming request and the knowledge base entries into meaningful vector representations. This model works by taking embeddings from the last hidden layer and averaging them out to get a single sentence-level embedding; this balances efficiency and semantic richness quite nicely.

For any input text, API request, or knowledge base entry, the system computes its embeddings by passing it into the transformer model: with attention-based encoding over a tokenized input, a comprehensive semantic representation is produced. These embeddings form the basis of generating output and comparing the similarities documented in API behaviors.

The degree of similarity between the generated and documented API responses is measured using cosine similarity. Cosine similarity refers to the angular distance between two high-dimensional vectors that calculates a normalized score ranging from −1 to 1. Higher scores mean higher semantic overlap and alignment.

The system's knowledge base is a structured repository containing prompt-response pairs derived from API documentation. Each entry includes:

- **Prompt**: A textual description of an API request.
- **Response**: The expected output as documented in the API specifications.

The system iteratively searches the knowledge base for the most semantically relevant entry with respect to the given API request. Comparing the entry embeddings, the system chooses the entry whose similarity is the highest, thus making a response contextually appropriate.

The textual input provided undergoes extensive preprocessing in order to be compatible with the transformer model. This text preprocessing includes:

- Tokenization of the input text into subword units.
- Padding and truncation to maintain uniformity in input lengths.
- Inclusion of special tokens for marking sequence boundaries.

This preprocessing pipeline guarantees the consistency of embedding generation, ensuring appropriate semantic comparisons.

The architecture scale the system is designed for would accommodate growth of knowledge bases without any performance degradation. Its embedding-based retrieval mechanism makes rapid calculations over similarity easily enabled, which helps the honeypot effectively scale up as new API endpoints and interactions are included with this setup. This will ensure the adaptability of the system toward evolving attack vectors.

4.4. Evaluation of similarity scores

Similarity score evaluation plays a critical role in ensuring semantic alignment between the honeypot-generated responses and the legitimate API behaviors. This process measures the fidelity of the generated outputs by quantifying their alignment with documented responses, hence validating the system's capability to produce realistic interactions. The similarity scores are computed using cosine similarity, a mathematical approach that determines angular closeness between two high-dimensional vector representations of text.

To calculate these scores, the generated response and the documented API response are first passed through the transformer-based embedding model to obtain their dense vector representations, which capture the semantic relationships between words and phrases. The embeddings for each text are extracted by averaging the token embeddings from the last hidden layer of the transformer, yielding one vector that represents the meaning of the whole text. This embedding step helps the system capture subtle context that is important for evaluating API behavior.

Once embeddings have been generated, the next step is to calculate cosine similarity. This metric calculates the cosine of the angle between two vectors, returning a scalar value within the range of −1 to 1. The closer the value is to 1, the higher the semantic similarity. A value close to 0 means there is no significant relation. In this application, negative similarity scores are few and generally represent contrasting meanings, which are unlikely in API response generation. The formula for this calculation will involve the dot product of the two vectors in the numerator, divided by the magnitude of the two vectors multiplied together in the denominator. This normalizes and ensures that the similarity is scale-independent of the absolute embeddings.

Similarity test includes the comparison between the embeddings of the generated response and pre-computed embedding of all entries in the knowledge base. Comparisons give a similarity score, and the one

with the best score among all entries will be most relevant match. There is a filtering via a certain threshold for quality. The threshold usually ranges between 0.85 and 0.95, depending on the complexity of the API and the domain-specific requirements. Responses scoring below the threshold are always flagged for further analysis, which may include manual changes or retraining of the model with more data.

It is also designed to handle edge cases that would come up during the analysis of similarity. Ambiguous API requests with incomplete parameters, for example, may lead to embeddings with low specificity, which could impact the similarity scores. If there is no strictly correct answer, it selects a semantically most plausible match based on the context. When more than one entry in the knowledge base gives a high score, the system utilizes auxiliary criteria to select an entry: either the response length or structural congruence is used to pick out the best match. Such auxiliary assessments increase the reliability of the system when confronted with difficult cases.

All the entries in the knowledge base are pre-computed and indexed systematically in this model to reduce the computational load at runtime while comparing similarities. It exploits vector operations like the dot product and computation of vector norms for batch processing to reduce latency in handling large knowledge bases. The corresponding optimization here is through dynamic normalization—that is, the threshold values are dynamically set according to the score distribution of each API type. This makes response validation consistent for a large number of different interactions, as seen in IoT smart home scenarios. Taken together, these technical methods help to reach the required levels of accuracy and reliability necessary to generate API responses within the honeypot environment that are misleading but still convincingly realistic in appearance.

5. Implementation and deployment

5.1. System deployment

The honeypot's web API endpoint mimics real web APIs and works with multiple network infrastructures and API management systems. Many endpoint architectures and API versions are used to impersonate legitimate honeypot API services for different request types and protocol standards. JSON and XML data formats, rate limits, authentication, and HTTP response codes are supported by the endpoints.

The honeypot API mimics public API functionality and follows endpoint naming rules to increase its credibility. Due to its ability to flexibly adapt to numerous API versions, attackers have trouble identifying the honeypot from real APIs. LLM examines each request and generates a synthetic answer based on the knowledge base's real API documentation prompt-response pairs. A well-crafted decoy ensures an API-expected answer.

The containerization and orchestration solution scales and manages many honeypot instances across cloud and on-premises networks. Honeypots can be segregated and replicated in containers for quick deployment across networks with minimal configuration. Kubernetes orchestrates honeypot instance scaling based on traffic and demand. The system's flexibility and modularity allow quick API and endpoint configuration adjustments to comply with new attack methods and actual service changes. The honeypot and other online API endpoints are designed to integrate seamlessly with network infrastructures and API management systems to look like real APIs. This deployment method mimics honeypot API services by using numerous endpoint architectures and API versions to handle different request types and protocol standards. These endpoints support JSON and XML data and have rate-limiting, authentication, and HTTP response codes.

The honeypot API mimics public API functionality and follows endpoint naming rules to increase its credibility. Due to its ability to flexibly adapt to numerous API versions, attackers have trouble identifying the honeypot from real APIs. The LLM examines each request and simulates a response using the knowledge base's API documentation's prompt-response pairings. A well-crafted decoy matches API behavior. The methodology scales and manages many honeypot instances across cloud and on-premises networks using containerization and orchestration. Containers provide for isolated and reproducible honeypots, enabling rapid network deployment with little configuration. Kubernetes orchestrates honeypot instance scaling based on traffic and demand. The system's flexibility and modularity allow quick API and endpoint configuration adjustments to comply with changing attack techniques and legal service changes.

5.2. Logging and monitoring

Honeypots require extensive logging and monitoring to respond quickly and analyze data over time. For future research, all honeypot interactions—questions and responses—are recorded. The originating IP address, request timestamp, user-agent details, and API endpoint are the most relevant metadata in these logs. This metadata provides a complete dataset for attack pathway and activity analysis.

The required data helps the system recognize strange trends that indicate automated probing or brute-force attacks. Frequency, kind, and source are metrics. Logs highlight patterns and trends related to prospective attacks, such as targeted endpoint exploitation and API probing. The honeypot system alerts for unusual activity in real time by identifying and classifying repetitive or atypical access patterns. These warnings can educate security teams, prompt automatic replies, and improve IP address and user agent recording and monitoring.

Logs are regularly evaluated to improve knowledge base and answer correctness and coherence. As attackers become more sophisticated, this recurrent function creates answers that match authentic API activity, maintaining the honeypot's deception method. Historical data can also improve prompt-response pairings and LLM API compliance. Security Information and Event Management (SIEM) technologies employ the existing logging architecture to aggregate and analyze log data with other security monitoring data. Honeypot installations integrate with SIEM platforms to detect and respond to network events and threat intelligence feeds in real time. Logged data can be utilized to build threat analysis machine learning models. This automates attack pattern recognition and strengthens the honeypot's defenses against increasingly advanced adversaries.

This extensive logging and monitoring architecture improves the honeypot by revealing danger environments, enabling proactive security, and improving response generation. This honeypot can fool and provide insights for threat assessment and defensive strategy improvement due to its thorough documentation of all essential information.

6. Evaluation and results

6.1. Similarity evaluation

In order to validate the accuracy and authenticity of the outputs generated by the decoy web API honeypot, a two-level similarity analysis approach has been followed. This process ensures that the outputs so generated are very similar to real API interactions using known prompt-response pairs as baselines. Request Similarity Matching is the first step in this approach, whose goal is to identify the most relevant prompt-response pair from a dataset, based on the similarity between the incoming request and the stored prompts. When the system receives a request, it calculates a measure of its similarity relative to all prompts stored in the prompt-response dataset. The evaluation can be performed with various measures of similarity, such as cosine similarity or semantic similarity based on embeddings, which represent contextual correspondence between the incoming request and the archived prompts very effectively. The trigger with the highest value of similarity is selected from them, and its corresponding response is used as the reference for both the response production and evaluation. It then assures the decoy system obtains a reference response that optimally matches the intent

Table 1
Summary of sample requests.

Request Method	Request Similarity	Reference Method	Reference Response	Generated Response	Response Similarity
GET	0.9887	GET	200 OK; {'transactions': {'transactionId': 'TRANS001′, 'amount': 240,000}}	200 OK; {'transactionId': 'TXN003′, 'amount': 25.00, 'type': 'transfer', 'recipientId': 'USER002′}	0.9440
POST	0.9937	POST	201 Created; {'patientId': 'PAT031′, 'status': 'Patient added successfully'}	201 Created; {'patientId': 'PAT031′, 'status': 'Patient added successfully'}	1.0000
DELETE	0.9989	DELETE	200 OK; {'studentId': 'STU043′, 'status': 'Student deleted successfully'}	200 OK; {'studentId': 'STU041′, 'status': 'Student deleted successfully'}	0.9992
GET	0.9987	GET	200 OK; {'studentId': 'STU007′, 'name': 'George Blue', 'age': 20, 'gender': 'male', 'contact': '555–0101′, 'enrollmentYear': 2023}	200 OK; {'studentId': 'STU008′, 'name': 'Hannah Gray', 'age': 21, 'gender': 'female', 'contact': '555–3030′, 'enrollmentYear': 2022}	0.9932
DELETE	0.9990	DELETE	200 OK; {'studentId': 'STU030′, 'status': 'Student deleted successfully'}	200 OK; {'studentId': 'STU022′, 'status': 'Student deleted successfully'}	0.9983

and context of the incoming question.

During the second stage, Response Similarity Evaluation, the quality of the generated response is checked against the real response in the selected prompt-response pair. The goal of this verification is to check how the generated response matches the expected behavior of the API as recorded in the reference dataset. Through the similarity calculation of the generated and real response to the prompt-response pair, the system ensures that the generated output truly reflects valid API behavior. High scores in similarity represent more exact congruence, implying that the decoy API is indeed mimicking the real responses. The process grows credibility for the honeypot, which is effective in logging realistic interactions with attackers and deriving valuable information on potential threats.

6.2. Results

We evaluate the performance of the decoy web API honeypot in generating realistic and contextually accurate responses. The evaluation is based on a two-step similarity analysis, involving (1) the identification of the most similar prompt-response pair for each incoming request and (2) the comparison between the generated response and the actual response associated with the matched prompt-response pair. Table 1 provides a summary of sample requests, detailing the request similarity scores, reference responses, generated responses, and response similarity scores.

For every incoming request, the system calculates a Request Similarity score, matching it against previously stored ones in the dataset, to select the prompt-response pair having the highest degree of match. This selection shows the systems' ability to correctly single out contextually relevant prompts. As a case in point, the GET request with a similarity score of 0.9887 was matched with a saved request that fetches transaction details, demonstrating that the system was able to recognize and align the context of the request in the dataset accurately. The Response Similarity score quantifies how much the generated response corresponds to the reference response from the relevant prompt-response pair, ensuring that the simulated API functionality in the context of IoT home automation is accurate. The system correctly emulates expected API outputs, as evidenced by the similarity scores of 1.0000 for POST adding a new smart device and 0.9992 for DELETE removing a user's profile. These results confirm this system in its capability of creating realistic yet contextually exact decoy responses to impersonate real API interactions aimed at deceiving attackers and gathering meaningful intelligence.

An even better illustration of the honeypot's efficacy in response generation is demonstrated by the average response similarity score of 0.9780. Such a high average score indicates a system with a reliable ability to generate responses that closely match the expected results, which would help it serve more effectively as a powerful deception mechanism in cybersecurity. By generating replies that are plausible and contextually relevant, the decoy web API can go much deeper in its ability to interact with potential attackers, thus revealing a great deal about their tactics and motivations.

7. Conclusion

This work describes the design and evaluation of a decoy Web API honeypot based on state-of-the-art language models with Retrieval-Augmented Generation, for Deriving believable answers and enlarging its potential of being an effective decoy in the context of cybersecurity. The architecture is able to obtain contextually appropriate and plausible responses by adaptively matching incoming requests to the established prompt-response pairs and evaluating response accuracy by applying similarity metrics.

These results show that the honeypot is capable of actively interacting with potential attackers, as indicated by the high request and response similarity scores. With a mean response similarity value of 0.9780, the system proves capable of closely mimicking expected API behaviors, thereby rendering an authentic user experience that encourages a longer stay by users. It has a high level of realism, which not only deceives the adversaries but also allows gathering important insights about their strategies and methodologies for improving defensive capabilities. This research delineates the potential benefits of using LLM-based honeypots in the cybersecurity domain to protect critical systems by redirecting aggressors away from actual resources and monitoring their actions in a controlled environment. Future work will continue to diversify the dataset of prompt-response pairs, allowing for a larger range of API interactions. Another promising avenue is increasing the model's real-time adaptability in order to detect even more complex forms of malicious behavior. Therefore, integrating state-of-the-art artificial intelligence into honeypot systems is considered a very promising direction forward, shaping the future of advanced and proactive cyber defense solutions.

CRediT authorship contribution statement

Anıl Sezgin: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Methodology, Conceptualization. **Aytuğ Boyacı:** Writing – original draft, Validation, Supervision, Methodology.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Anil Sezgin reports article publishing charges was provided by Siemens Turkey. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We would like to thank Siemens Turkey for their support in the

completion of the study.

Data availability

The authors do not have permission to share data.

References

Angeli, T., Reti, D., Schneider, D., Schotten, H., 2024. False flavor honeypot: deceiving vulnerability scanning tools. In: IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). Vienna, Austria.

Bartwal, U., Mukhopadhyay, S., Negi, R., Shukla, S., 2022. Security orchestration, automation, and response engine for deployment of behavioural honeypots. In: IEEE Conference on Dependable and Secure Computing (DSC). Edinburgh, United Kingdom.

Buzzio-Garcia, J., 2021. Creation of a high-interaction honeypot system based-on docker containers. In: Fifth World Conference on Smart Trends in Systems Security and Sustainability (WorldS4). London, United Kingdom.

Commey, D., Hounsinou, S., Crosby, G.V., 2024. Strategic deployment of honeypots in blockchain-based IoT systems. In: IEEE International Conference on AI Circuits and Systems (AICAS). Abu Dhabi, United Arab Emirates.

Conti, M., Trolese, F., Turrin, F., 2022. ICSpot: a high-interaction honeypot for industrial control systems. In: International Symposium on Networks, Computers and Communications (ISNCC). Shenzhen, China.

Dang, F., Yan, L., Yang, Y., Li, S., Han, L., 2024. Research on network intrusion detection technology based on honeypot. In: International Conference on Information Science, Parallel and Distributed Systems (ISPDS). Guangzhou, China.

Ding, W., 2024. A GAN-based security strategy for WSN networks based on honeypot algorithm. Phys. Commun. 1–9 cilt 62.

Gao, Y., Yang, L., Zhu, R., Yang, F., Cao, Y., Zhang, L., 2024. Intelligent hopping mechanism for deception defense scenarios based on reinforcement learning. In: Conference on Local Computer Networks (LCN). Normandy, France.

Gillick, D., Kulkarni, S., Lansing, L., Presta, A., Baldridge, J., Ie, E., Garcia-Olano, D., 2019. Learning dense representations for entity retrieval. In: Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL), pp. 528–537.

Hu, Y., Cheng, S., Ma, Y., Chen, S., Xiao, F., Zheng, Q., 2024. MySQL-pot: a LLM-based honeypot for MySQL threat protection. In: International Conference on Big Data Analytics (ICBDA). Tokyo, Japan.

Jiang, K., Zheng, H., 2020. Design and implementation of a machine learning enhanced web honeypot system. In: International Congress on Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI). Chengdu, China.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Chen, D., Yih, W., 2020. Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781.

Kristyanto, M.A., Studiawan, H., Pratomo, B.A., 2022. Evaluation of reinforcement learning algorithm on SSH honeypot. In: International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE). Yogyakarta, Indonesia.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: NIPS'20: Proceedings of the 34th International Conference on Neural Information Processing Systems, pp. 9459–9474.

Li, J., Tang, T., Zhao, W.X., Nie, J., Wen, J., 2024. Pre-trained language models for text generation: a survey. ACM. Comput. Surv. 56 (9), 1–39.

Liatifis, A., Eleftheriadis, C., Mpatzos, Z., Nanos, I., Lagkas, T., Goudos, S., Argyriou, V., Psannis, K.E., Moscholios, I.D., Sarigiannidis, P., 2024. SiHoneypot: a digital twin-based honeypot for autonomous vehicles. In: International Conference on Modern Circuits and Systems Technologies (MOCAST). Sofia, Bulgaria.

Loevenich, J.F., Adler, E., Mercier, R., Velazquez, A., Lopes, R.R.F., 2024. Design of an autonomous cyber defence agent using hybrid AI models. In: International Conference on Military Communication and Information Systems (ICMCIS). Koblenz, Germany.

Lopez-Yepez, J.S., Fagette, A., 2022. Increasing attacker engagement on SSH honeypots using semantic embeddings of cyber-attack patterns and deep reinforcement learning. In: IEEE Symposium Series on Computational Intelligence (SSCI). Singapore, Singapore.

Lygerou, I., Srinivasa, S., Vasilomanolakis, E., Stergiopoulos ve, G., Gritzalis, D., 2022. A decentralized honeypot for IoT protocols based on Android devices. Int. J. Inf. Secur. 1211–1222 cilt 21.

Maldonado, M., Johnson, C., Gulati, M., Jaspers, T., Roysdon, P.F., 2024. Advanced Cyber Deception Framework (ACDF): a comprehensive study. In: International Conference on Computing, Networking and Communications (ICNC). Big Island, HI, USA.

Miao, S., Li ve Q. Pan, Y., 2023. Honeypot game theory against DoS attack in UAV cyber. Comput., Mater. Continua (CMC) (3), 2745–2762 cilt 76.

Qin, X., Jiang, F., Cen ve, M., Doss, R., 2023. Hybrid cyber defense strategies using Honey-X: a survey. Comput. Netw. 1–17 cilt 230.

Rashid, Z.U., Haq, A., Hasan, S.T., Furhad, H., Ahmed ve, M., Ullah, A.B., 2024. Faking smart industry: exploring cyber-threat landscape deploying cloud-based honeypot. Wireless Networks 4527–4541 cilt 30.

Raut, U., Nagarkar, A., Talnikar, C., Mokashi, M., Sharma, R., 2023. Engaging attackers with a highly interactive honeypot system using ChatGPT. In: International Conference On Computing, Communication, Control And Automation (ICCUBEA). Pune, India.

Sharma, D., Singh, A., Chitkara, S., Sharma, T., 2023. Honeypot networks in deception technology for IOT devices. In: International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT). Faridabad, India.

Sladić, M., Valeros, V., Catania, C., Garcia, S., 2024. LLM in the shell: generative honeypots. In: IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). Vienna, Austria.

Suhail, S., Iqbal, M., Hussain ve, R., Jurdak, R., 2023. ENIGMA: an explainable digital twin security solution for cyber–physical systems. Comput. Ind. 1–16 cilt 151.

Tang, J., Chen, M., Chen, H., Zhao ve, S., Huang, Y., 2023. A new dynamic security defense system based on TCP_REPAIR and deep learning. J. Cloud Comput. 1–21 cilt 12.

Tian, W., Ji, X., Liu, W., Liu, G., Lin, R., Zhai ve, J., Dai, Y., 2019. Defense strategies against network attacks in cyber-physical systems with analysis cost constraint based on honeypot game model. Comput., Mater. Continua (CMC) (1), 193–211 cilt 60.

Zhou, X., Almutairi, L., Alsenani ve, T.R., Ahmad, M.N., 2023. Honeypot based industrial threat detection using game theory in cyber-physical system. J. Grid. Comput. (59), 1–11 cilt 21.